

Specifikacija Projekta

Softverski agenti – 2026

Student: Lazar Nestorović

Broj indeksa: RA 145/2022

Datum: maj 2026.

1. Tim

Ime i prezime	Broj indeksa	Grupa
Lazar Nestorović	RA 145/2022	3

Projekat se radi samostalno. Kao jedini član tima, student implementira minimum četiri aktora i dva dodatna elementa aktorskog okruženja (klasterovanje se računa kao dva).

2. Zadatak

Ciljna ocena: 10 — obuhvata sve zahteve za ocene 7 i 9:

- Ocena 7** Federativno učenje + CRDT + klasterovanje

- **Ocena 9** Aktorski radni okvir (osnovne + 2 dodatne funkcionalnosti)
- **Ocena 10** Kombinacija svega navedenog

Projekat se bavi implementacijom **distribuiranog sistema za federativno učenje u domenu predikcije cena nekretnina**, baziranog na generičkom aktorskom radnom okviru razvijenom u programskom jeziku **Go**. Sistem omogućava da više finansijskih institucija (banaka/agencija) zajednički trenira model linearne regresije, bez međusobne razmene privatnih podataka o klijentima.

3. Federativno učenje

3.1 Problem koji se rešava

Više finansijskih institucija (npr. banaka i agencija za nekretnine) želi da zajednički poboljša predikciju cena stanova i kuća. Svaka institucija poseduje sopstveni skup podataka o transakcijama koji je poverljiv i ne sme biti deljen sa ostalim učesnicima. Cilj je konstruisati **globalni prediktivni model** koji je bolji od modela treniranih samo na lokalnim podacima, a da privatnost ostane sačuvana.

3.2 Algoritam: Federated Averaging (FedAvg)

Koristiće se algoritam **FedAvg** (McMahan et al., 2017) — industrijski standard za federativno učenje. Svaki klijentski čvor trenira lokalni model gradijentnim spustom, nakon čega server agregira parametre ponderisanim prosekom:

Koraci algoritma FedAvg:

1. Koordinator šalje inicijalne (ili tekuće) parametre globalnog modela svim klijentima
2. Svaki klijent trenira lokalni model na sopstvenim podacima kroz E epoha (lokalni SGD)
3. Klijenti šalju ažurirane parametre (gradijente ili težine) koordinatoru
4. Koordinator agregira parametre: $w_{\text{global}} = \sum (n_k / N) * w_k$, gde je n_k broj uzoraka klijenta k
5. Ponavlja se kroz R rundi dok model ne konvergira

Model: Višestruka linearna regresija (može se proširiti na plitku neuronsku mrežu — MLP sa 1-2 skrivena sloja) implementirana u Go bez eksternih ML biblioteka.

3.3 Skup podataka

Svojstvo	Opis
Dataset	California Housing Dataset (UCI / Kaggle) — 20,640 uzoraka
Features	Medijana prihoda, starost kuće, broj soba, broj spavaćih soba, populacija, geografska lokacija (lat/lon)
Target	Medijana cene kuće (u USD)
Distribucija	Podaci se dele na N particija (po jedna po klijentu). Podržati i IID i non-IID podelu (po geografskim oblastima).
Preprocessing	Normalizacija min-max, imputacija nedostajućih vrednosti, uklanjanje autlejera

3.4 Distribucija treniranja

- **Topologija:** Star arhitektura (koordinator + N klijenata), uz P2P klaster za napredni mod
- **Klijenti:** 3–5 klijentskih čvorova koji simuliraju različite finansijske institucije
- **Komunikacija:** gRPC između udaljenih aktera (actor framework sloj)
- **Bezbednost:** Agregirani gradijenti nikada ne otkrivaju sirove podatke
- **Klasterovanje klijenata:** Klijenti se grupišu prema sličnosti distribucije podataka (K-Means na feature statistikama) — heterogeni klijenti treniraju klaster-specifične modele pre globalne agregacije

3.5 CRDT integracija

Upotreba CRDT struktura:

- **G-Counter** — brojanje završenih rundi treniranja po čvoru (bez konflikata pri agregaciji)
- **LWW-Register (Last-Write-Wins Register)** — čuvanje poslednje verzije globalnog modela (timestamp-based merge)
- **OR-Set (Observed-Remove Set)** — skup aktivnih klijentskih čvorova u klasteru (dodavanje/uklanjanje bez konflikata)

- **G-Set** — akumulacija metrika evaluacije tokom rundi (nepromenjivi log)

CRDT strukture omogućavaju da čvorovi ažuriraju stanje asinhorno i da se automatski pomire bez centralnog koordinatora ili zaključavanja.

3.6 Evaluacija rezultata

Metrika	Opis
MSE / RMSE	Osnovna metrika greške predikcije cena
MAE	Srednja apsolutna greška — interpretabilna u dolarima
R² skor	Koeficijent determinacije — koliko varijanse model objašnjava
Konvergencija	Grafikon gubitka po rundi (federalni model vs. lokalni modeli)
Poređenje	Federalni model vs. lokalni model vs. centralizovani model (baseline)

4. Aktori

4.1 Vrste aktora i uloge

Aktor	Uloga	Implementira student
CoordinatorActor	Upravlja rundama federativnog učenja: inicijalizuje globalni model, pokreće runde, prikuplja ažuriranja, pokreće agregaciju, distribuira novi model	✓
ClientActor	Predstavlja jednu finansijsku instituciju: prima globalni model, trenira lokalni model na privatnim podacima, šalje gradijente/težine koordinatoru	✓
AggregatorActor	Izvršava FedAvg agregaciju primljenih parametara od klijenata, izračunava ponderisani prosek, vraća ažurirani globalni model koordinatoru	✓
MonitorActor	Prati napredak treniranja: sakuplja metrike (loss, RMSE, MAE), generiše izveštaje, upozorava koordinatora na stagnaciju	✓
ClusterManagerActor	Grupiše klijente prema sličnosti podataka (K-Means), dodeljuje klaster koordinatore, upravlja klaster topologijom	✓
SupervisorActor	Nadzire sve aktore u sistemu: detektuje greške, implementira	✓

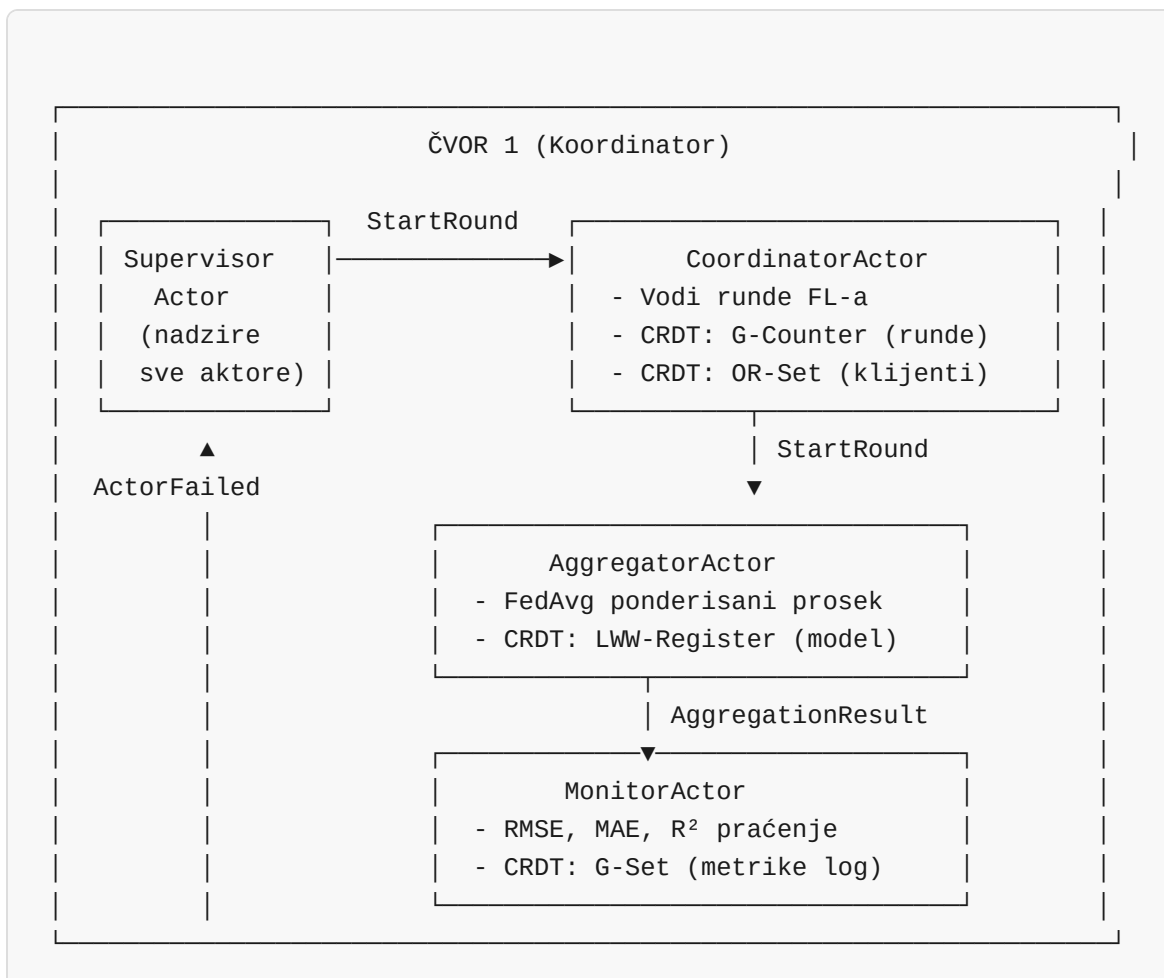
	strategije restarta (one-for-one, exponential backoff)	
--	--	--

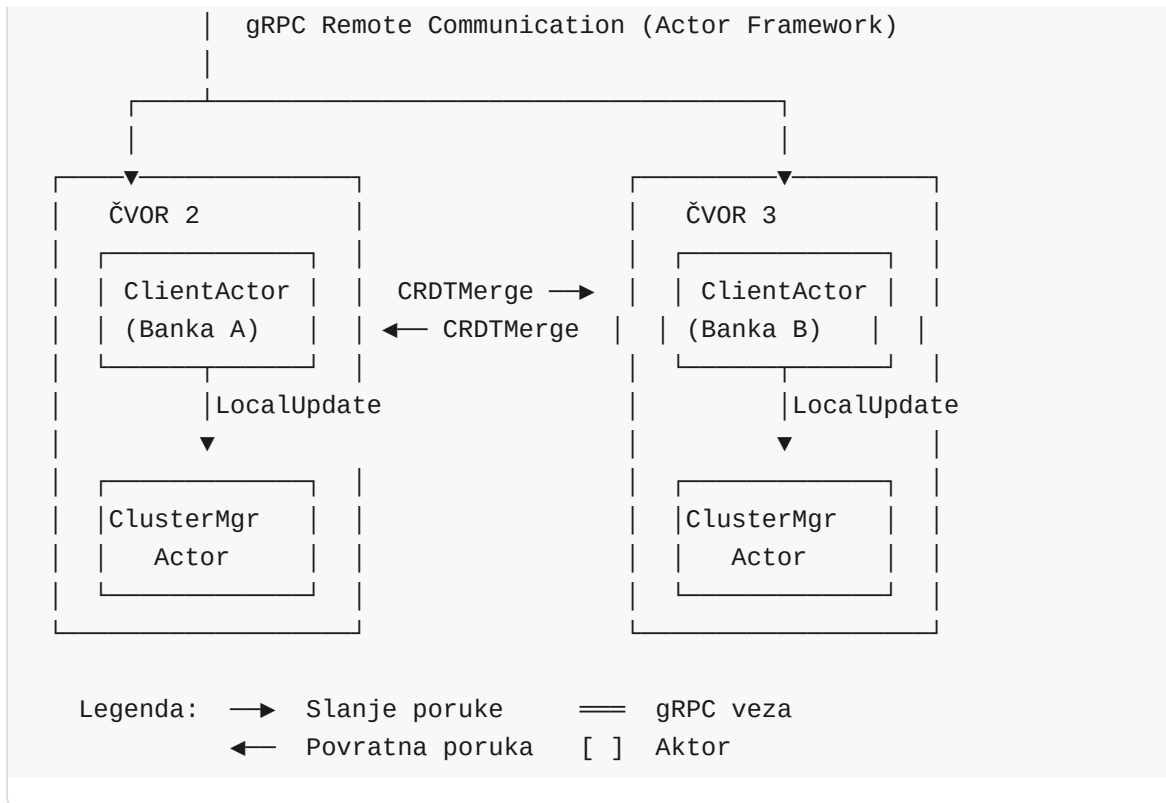
4.2 Poruke između aktora

Poruka	Pošiljalac → Primalac	Sadržaj	Namena
StartRound	Coordinator → Client*	round_id, model_weights[], learning_rate, epochs	Pokretanje nove runde lokalnog treniranja
LocalUpdate	Client → Aggregator	client_id, round_id, gradients[], num_samples, loss	Slanje lokalnih ažuriranja modela
AggregationResult	Aggregator → Coordinator	round_id, global_weights[], aggregated_loss	Rezultat FedAvg agregacije
RoundComplete	Coordinator → Monitor	round_id, global_loss, num_clients, elapsed_time	Izveštaj o završenoj rundi za praćenje
MetricsReport	Monitor → Coordinator	round_id, rmse, mae, r2_score, convergence_flag	Evaluacione metrike, signal za zaustavljanje
ClusterAssign	ClusterManager → Client*	cluster_id, cluster_members[], cluster_coordinator_id	Dodela klijenta klasteru

ActorFailed	Runtime → Supervisor	actor_id, actor_type, error_msg, restart_count	Notifikacija o padu aktora
RestartActor	Supervisor → Runtime	actor_id, strategy (one-for-one/all), delay_ms	Nalog za restart neuspešnog aktora
CRDTMerge	Any → Any	crdt_type, payload (serialized CRDT state)	Sinhronizacija CRDT stanja između čvorova

4.3 Dijagram komunikacije aktora





5. Implementacija aktorskog radnog okruženja

Radni okvir je **generički** i ne zavisi od logike federativnog učenja. Može se koristiti za bilo koji distribuirani sistem baziran na aktorskom modelu.

5.1 Obavezni elementi

Element	Opis implementacije
Aktori	Svaki aktor je Go goroutine sa jedinstvenim <code>ActorID</code> (UUID). Interface <code>Actor</code> definiše metode <code>Receive(msg Message)</code> i <code>OnStart(ctx) / OnStop()</code> .
Asinhrono slanje i primanje poruka	Poruke se šalju neblokirajući (<code>ActorRef.Tell(msg)</code>). Čitanje iz sanduča je asinhrono u pozadinskoj goroutini aktora.
Sanduče (Mailbox)	Baferisani Go kanal (<code>chan Message</code>) po aktoru. Podržati <i>bounded</i> i <i>unbounded</i> mailbox, sa FIFO i prioritetskom strategijom.
Menjanje ponašanja (stanja) aktora	<code>context.Become(newBehavior BehaviorFunc)</code> — aktor može da zameni svoju <code>Receive</code> funkciju u toku izvršavanja (state machine pattern). Stack ponašanja sa <code>Unbecome()</code> .
Lifecycle događaji (stvaranje i otkazivanje)	Sistemske poruke <code>Started</code> , <code>Stopping</code> , <code>Stopped</code> , <code>Restarting</code> . Hook-ovi <code>OnPreStart</code> , <code>OnPostStop</code> , <code>OnPreRestart</code> .
Udaljena komunikacija aktora	gRPC transport sloj. <code>RemoteActorRef</code> transparentno prosleđuje poruke udaljenom aktoru (serialization putem Protocol Buffers). Podržati <code>Tell</code> (fire-and-forget) i <code>Ask</code> (request-response sa timeout-om).

5.2 Dodatni elementi

Implementirane dodatne funkcionalnosti (klasterovanje = 2 od 2 potrebnih):

5.2.1 Dodatni 1+2 Klasterovanje aktora

(Računa se kao dve dodatne funkcionalnosti)

- **Service Discovery:** Gossip protokol za otkrivanje čvorova u klasteru (inspirisan Akka Cluster). Čvorovi periodično razmenjuju listu poznatih članova (*heartbeat*).
- **Distribuirani registar aktora:** Konzistentno hash prsten (consistent hashing) za raspodelu aktora po čvorovima. Svaki čvor zna ko je odgovoran za koji skup ActorID-eva.
- **Cluster-aware ActorRef:** Transparentno rutiranje — `system.ActorOf(props, "naziv")` automatski kreira aktor na najpogodnijem čvoru.
- **Singletons i distribuirani Pub/Sub:** Tačno jedan primerak *singleton* aktora u celom klasteru; event bus za broadcast poruka po temama.
- **Failover:** Automatska redistribucija aktora sa nedostupnog čvora.

5.2.2 Bonus Supervizija aktora

- **Hijerarhija supervizora:** Svaki aktor ima roditeljskog supervizora. Greška se propagira naviše kroz hijerarhiju (*error kernel pattern*).
- **Strategije restarta:**
 - `OneForOne` — restartuje samo neuspešni aktor
 - `OneForAll` — restartuje sve potomke supervizora
 - `ExponentialBackoff` — progresivno odlaganje restarta
- **Escalation:** Ako aktor premaši *max_restarts* u *window* vremenu, greška se eskalira roditeljskom supervizoru.

5.3 Detalji implementacije (tehnologije)

Komponenta	Tehnologija	Svrha
------------	-------------	-------

Programski jezik	Go 1.22+	Nativna podrška za goroutines i channels — idealno za aktorski model
Udaljena komunikacija	gRPC + Protocol Buffers	Efikasna binarna serijalizacija, bidirekciono streaming
Service Discovery	Custom Gossip protokol	Distribuirano otkrivanje i praćenje čvorova bez centralnog servera
CRDT implementacija	Ručna implementacija u Go	G-Counter, LWW-Register, OR-Set, G-Set
Konfiguracija	YAML + env variable	Konfiguracija broja klijenata, rundi, learning rate-a
Testiranje	Go testing + Testify	Unit testovi framework-a, integracioni testovi FL sistema
Dataset	California Housing (CSV)	Učitavanje i particija podataka po klijentima
Kontejnerizacija	Docker + Docker Compose	Simulacija više mašina lokalno; deploy na stvarnim mašinama
Praćenje performansi	Prometheus + Grafana	Metrike treniranja i performanse aktorskog sistema u realnom vremenu

5.4 Demonstracija funkcionalnosti

Tokom odbrane projekta biće demonstrirani:

1. **Lokalna demonstracija (Docker Compose):** Pokretanje 3–5 klijentskih čvorova i koordinatora na jednoj mašini u izolovanim kontejnerima
2. **Višemašinska demonstracija:** Deployment na dve fizičke/virtuelne mašine sa gRPC komunikacijom
3. **P2P mod:** Federativno učenje bez centralnog koordinatora — klijenti direktno razmenjuju CRDT stanje

4. **Provider klaster:** Koordinator upravlja registrom aktora i raspoređuje ih na čvorove
5. **CRDT sinhronizacija:** Demo konzistentnosti stanja pri istovremenim ažuriranjima
6. **Supervizija:** Simulacija pada klijentskog aktora i automatski restart
7. **Konvergencija modela:** Vizualizacija pada RMSE kroz runde FL-a

6. Struktura repozitorijuma

```

/
├── actor-framework/           ← Generički aktorski radni okvir
│   ├── actor.go              ← Actor interface i ActorRef
│   ├── mailbox.go            ← Mailbox implementacija
│   ├── context.go            ← ActorContext, Become/Unbecome
│   ├── system.go             ← ActorSystem (lifecycle management)
│   ├── remote/               ← gRPC transport sloj
│   ├── cluster/              ← Klasterovanje (gossip, consistent hash)
│   ├── supervision/          ← Supervisor hijerarhija i strategije
│   └── crdt/                  ← CRDT implementacije
├── federated-learning/        ← FL aplikacija (koristi actor-framework)
│   ├── actors/                ← CoordinatorActor, ClientActor, itd.
│   ├── model/                 ← Linearna regresija, FedAvg
│   ├── data/                  ← Učitavanje i particija dataseta
│   └── evaluation/            ← RMSE, MAE, R2 metrike
├── demo/                      ← Demo skripte i konfiguracije
├── dokumentacija/
│   └── SpecifikacijaProjekta.pdf
├── docker-compose.yml
└── README.md

```